BRIDGESTONE IT ENGINEERING

Platform Engineering — AI & Automation

ENTERPRISE SOLUTION ARCHITECTURE DOCUMENT

Bridgestone IT AI Assistant

Enterprise AI-Powered IT Service Management Platform

Document ID	BST-ESAD-2026-001
Version	1.0.0
Status	Released
Date	July 2026
Owner	Platform Engineering — AI & Automation
Audience	Engineering Leadership · Principal Engineers · Enterprise Architects
Classification	Internal — Engineering Leadership

Revision History

Version	Date	Author	Description
0.1	May 2026	Platform Engineering	Initial draft — AI pipeline and integration architecture
0.5	June 2026	Platform Engineering	Security, database and deployment chapters added
0.9	July 2026	Platform Engineering	Full review; ADR, observability and roadmap added
1.0	July 2026	Platform Engineering	Final release for engineering leadership review

Table of Contents

NOTE Open this document in Microsoft Word, then press Ctrl+A followed by F9 to populate the Table of Contents with page numbers.

3. Executive Summary

The Bridgestone IT AI Assistant is an enterprise-grade, AI-augmented IT Service Desk platform designed to reduce Level-1 support load, improve Mean Time To Resolution (MTTR), and preserve ServiceNow as the authoritative system of record.

The platform routes employee IT issues through a 23-node LangGraph agentic pipeline before escalating to human intervention. The AI agent conducts structured interviews, retrieves enterprise knowledge, executes diagnostic tools (VPN, Active Directory, Network), performs root-cause reflection, and — where self-service resolution is impossible — creates a properly enriched, validated ServiceNow Incident automatically.

A dual-approval workflow governs privileged software installation requests, routing through Manager approval and IT Admin grant operations without human triage. Four-role JWT-secured RBAC separates employee, manager, admin and superadmin concerns across dedicated portal surfaces.

IMPORTANT Platform Impact:

- Eliminates manual first-line triage for common IT issue categories
- Removes classification errors through AI-validated, schema-driven ServiceNow field mapping
- Provides real-time SLA visibility with automated seven-state escalation
- Delivers complete auditability through an append-only immutable audit trail

4. Business Problem

4.1 Context

Enterprise IT helpdesks operating at scale face a structural imbalance: ticket volume grows linearly with headcount, while Level-1 staffing is constrained by cost. At Bridgestone, this produces five measurable failure modes.

4.2 Failure Modes

Failure Mode	Root Cause	Business Impact
High L1 ticket volume	Manual triage of repetitive issues	Increased cost per ticket, slow MTTR
Incorrect ServiceNow classification	Human error in category / assignment group selection	Mis-routed tickets, SLA breach
Approval process latency	Manual manager notification for privileged requests	Delayed productivity recovery
SLA breach detection lag	Reactive monitoring; no automated escalation	SLA compliance risk, degraded experience
Lack of self-service resolution	No intelligent guidance before ticket	Unnecessary L1 escalation for solvable

	creation	issues

Table 1 — Business Failure Modes

4.3 Architectural Constraints

- ServiceNow must remain the system of record — no parallel ticket stores.
- Existing Microsoft ecosystem (Active Directory, Microsoft Graph, Outlook) must be respected.
- The solution must be deployable inside the Bridgestone network perimeter.
- AI output must never directly create side effects without deterministic validation.

5. Business Objectives

#	Objective	Measurement
O-1	Reduce L1 ticket volume	Percentage of issues resolved without ticket creation
O-2	Reduce MTTR	Average time from report to resolution
O-3	Improve ticket classification accuracy	ServiceNow field validation pass rate
O-4	Automate privileged software approval workflow	End-to-end time: request to access grant
O-5	Improve employee self-service experience	First-contact resolution rate
O-6	Maintain complete auditability	Audit log coverage across all state transitions
O-7	Preserve ServiceNow as system of record	All incidents traceable to a ServiceNow INC number
O-8	Enforce SLA compliance with automated escalation	Percentage of tickets escalated before SLA breach

Table 2 — Business Objectives

6. Solution Overview

6.1 Architecture Pattern

The platform is built on a Hybrid AI + Deterministic architecture. AI reasoning handles ambiguous, contextual decisions — intent detection, diagnostic planning, root-cause analysis, reflection and classification. Deterministic services handle all outcomes governed by business rules — approval routing, SLA computation, ServiceNow field mapping and state machine transitions.

This separation is deliberate. AI output is non-deterministic by nature. Every AI decision that produces a side effect (ticket creation, approval, access grant) is gated by deterministic validation before execution.

6.2 Platform Layers

Layer	Technology	Responsibility
Presentation Layer	Next.js 16 / React 19	Four role-separated portal surfaces (Employee · Manager · Admin · SuperAdmin)
API Gateway Layer	FastAPI + Uvicorn	Authentication, RBAC, rate limiting, security headers, PromptGuard
AI Orchestration	LangGraph StateGraph	23-node conditional AI pipeline with multi-step diagnostic loops
Deterministic Logic	Classification & SLA Services	Business-rule enforcement on all AI outcomes before system mutation
Integration Layer	ServiceNow, Graph, AD, VPN Adapters	Adapter-pattern enterprise system connectors with mock/live toggle
Data Tier	PostgreSQL 14 & Redis	ACID relational storage and metadata / session caching

Table 3 — Platform Architecture Layers

6.3 Platform Scope

In Scope	Out of Scope
L1 AI-driven issue diagnosis	L2/L3 deep infrastructure troubleshooting
ServiceNow incident creation and lifecycle	ServiceNow configuration and CMDB management
Privileged software request approval workflow	Software licensing management
SLA monitoring and automated escalation	ITSM capacity planning
RBAC-separated portal surfaces	HR and payroll integration
Audit logging for compliance	SOC/SIEM integration (roadmap)

Table 4 — Platform Scope

7. Functional Capabilities

7.1 Core Capabilities

Capability	Description
Conversational AI Triage	Employees describe issues in natural language; the agent conducts a structured diagnostic interview before escalating
Multi-Step Troubleshooting	The agent iteratively executes tool chains (VPN, AD, network) and loops until

	resolution or escalation
Automatic ServiceNow Incident Creation	Issues that cannot be self-resolved are classified, enriched, validated and submitted to ServiceNow via OAuth2
Dual-Approval Workflow	Privileged requests route through Manager approval, then Admin access grant, tracked in DB and ServiceNow
SLA Engine	APScheduler job evaluates open tickets every 60 seconds, progressing through seven escalation states
Role-Separated Portals	Four purpose-built surfaces for Employee, Manager, IT Admin and SuperAdmin roles
Analytics Dashboard	Ticket volume by category and team, priority breakdown, SLA compliance trend
Knowledge Base	Admin-managed KB articles with AI-assisted article suggestion from resolved ticket context

7.2 Platform-Level Capabilities

Capability	Implementation
Prompt Injection Protection	Three-tier risk classifier (HIGH → reject, MEDIUM → sanitise, LOW → warn) on every user message
Rate Limiting	Per-user/per-IP sliding-window limiter on /chat, /auth/login, /upload — no external dependencies
Immutable Audit Log	Append-only RBAC audit trail: SHA-256 payload hash, correlation ID, actor, ISO-8601 timestamp
Distributed Tracing	OpenTelemetry-compatible trace_span context manager instruments every pipeline node and LLM call
Structured JSON Logging	Per-request correlation ID propagated via Python contextvars through every log statement
Auto Schema Migration	Missing database columns detected and added on startup — no manual migration tooling required

8. Enterprise Business Workflow

8.1 Employee Issue Resolution Flow

The interaction sequence below details how an employee request moves from raw chat input through the security gateway, diagnostic graph, tool execution, and optional ServiceNow incident creation.

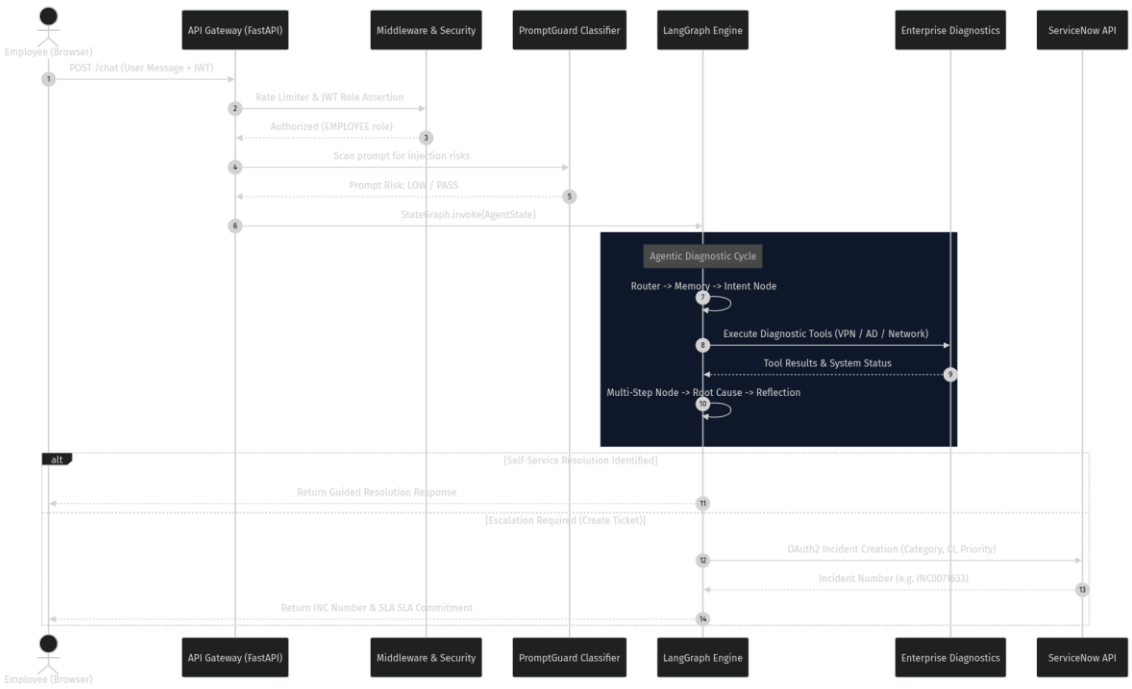

Figure 1 — Employee Issue Resolution — End-to-End Sequence Diagram

Figure 1 illustrates the end-to-end processing sequence for employee IT support interactions. Inbound HTTP requests traverse a mandatory FastAPI middleware security chain (Rate Limiter, JWT Auth, RBAC, and PromptGuard) before reaching the LangGraph StateGraph engine. Within the pipeline, diagnostic tools are invoked iteratively to gather evidence; if self-service resolution fails, an enriched ServiceNow incident is created via OAuth2 REST APIs, returning an official ticket number to the user.

8.2 Privileged Software Approval Workflow

Requests for privileged software (e.g. VS Code, SAP GUI, Adobe CC) follow a strict two-stage human-in-the-loop approval workflow before local access is granted.

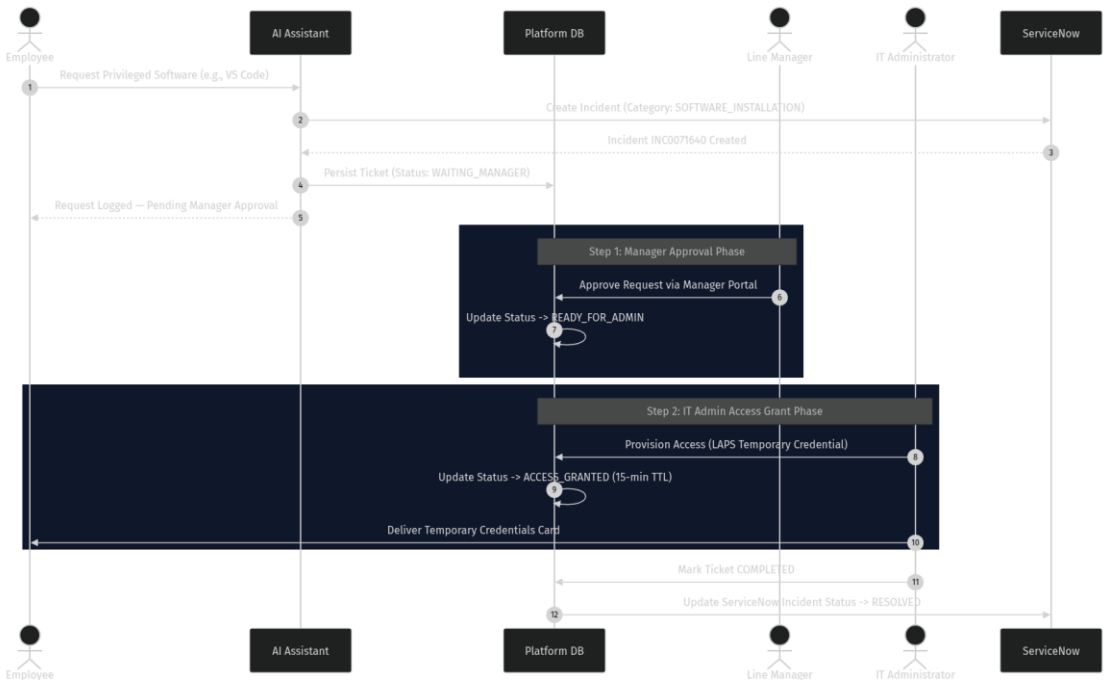

Figure 2 — Privileged Software Dual-Approval Workflow Sequence

Figure 2 depicts the state synchronization across the platform database, ServiceNow, Manager portal, and IT Admin queue for privileged requests. When an employee requests restricted software, the AI classifies the intent as a software installation requiring approval and immediately persists a ticket in WAITING_MANAGER status while notifying ServiceNow. Following Manager approval, the status transitions to READY_FOR_ADMIN, whereupon an IT Administrator issues temporary credentials (LAPS) and completes the ticket in ServiceNow.

8.3 SLA Escalation State Machine

The platform SLA engine evaluates open incidents every 60 seconds against defined operational level agreements.

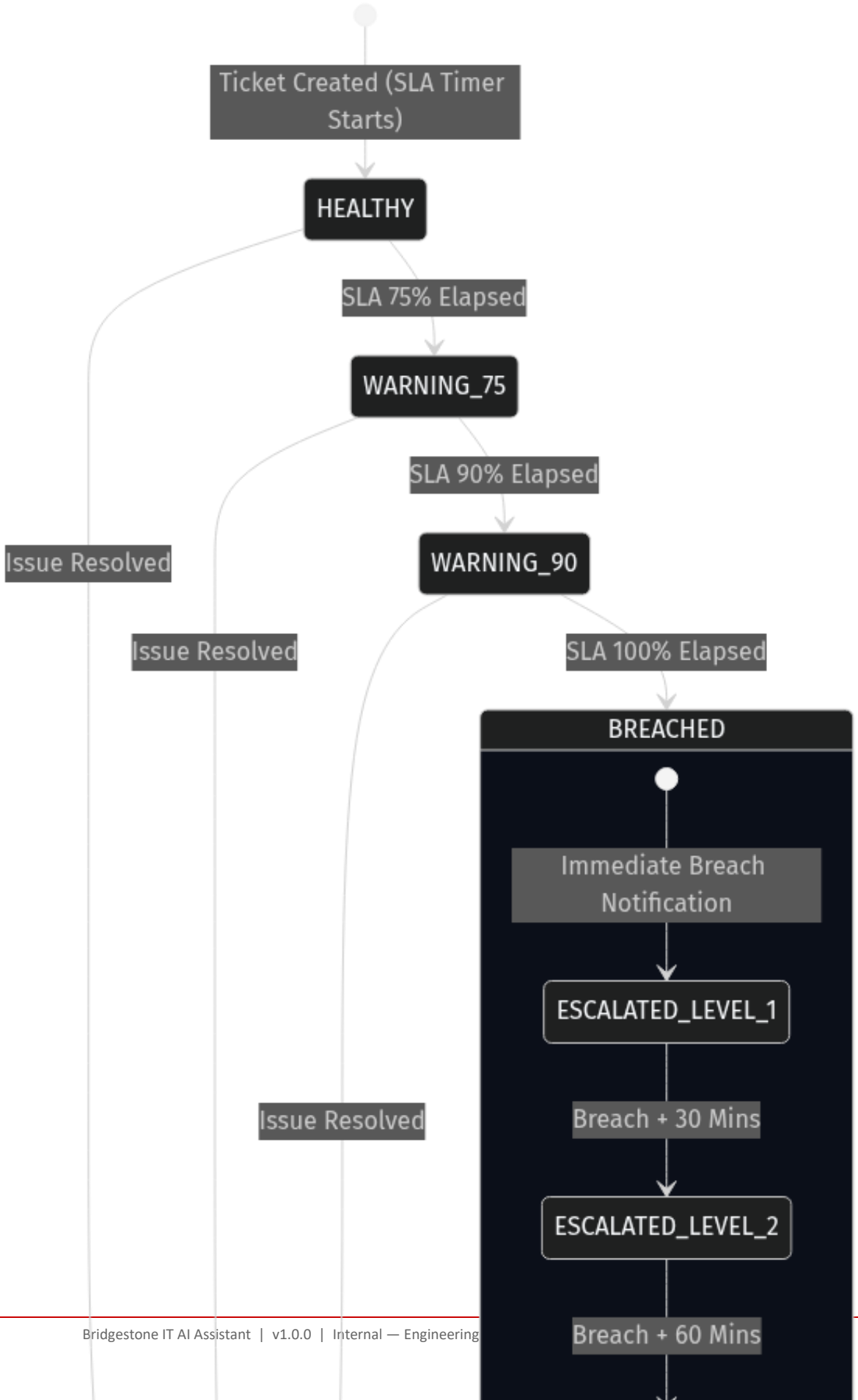
Ticket Created (SLA Timer Starts)
HEALTHY
SLA 75% Elapsed
WARNING_75
SLA 90% Elapsed
WARNING_90
SLA 100% Elapsed
BREACHED
Immediate Breach Notification
ESCALATED_LEVEL_1
Breach + 30 Mins
ESCALATED_LEVEL_2
Breach + 60 Mins
Issue Resolved
Issue Resolved
Issue Resolved

Figure 3 — SLA Escalation State Machine — Workflow States

Figure 3 details the seven-state SLA lifecycle transition matrix enforced by the background scheduler. Tickets originate in the HEALTHY state and progress through WARNING_75 (75% elapsed) and WARNING_90 (90% elapsed) triggers, notifying Managers and IT Administrators accordingly. If 100% of the SLA window elapses before resolution, the ticket transitions to BREACHED and automatically initiates Level-1, Level-2, and Level-3 management escalations at 30-minute intervals.

9. High-Level System Architecture

9.1 Architecture Overview

The Bridgestone IT AI Assistant platform is structured into clean architectural tiers ensuring complete isolation between presentation, gateway, agentic reasoning, deterministic logic, and persistent storage.

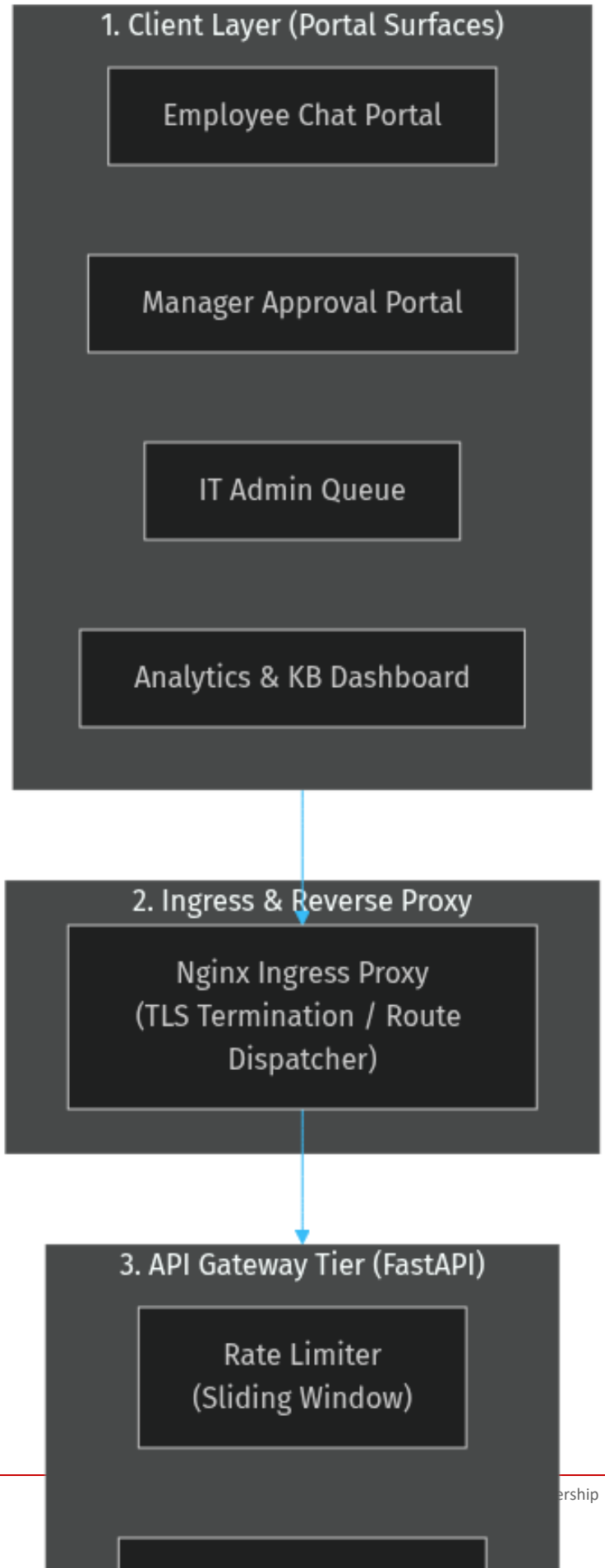
1. Client Layer (Portal Surfaces)
Employee Chat Portal
Manager Approval Portal
IT Admin Queue
Analytics & KB Dashboard
2. Ingress & Reverse Proxy
Nginx Ingress Proxy
(TLS Termination / Route
Dispatcher)
3. API Gateway Tier (FastAPI)
Rate Limiter
(Sliding Window)

Figure 4 — High-Level System Architecture — Component Topology

Figure 4 presents the macro architectural topology of the platform across seven isolated layers. Nginx handles intranet ingress and TLS termination, routing web traffic to Next.js portal surfaces and API requests to FastAPI gateway middleware. Business logic is cleanly segmented between LangGraph agentic reasoning, deterministic validation services, enterprise integration adapters (ServiceNow, Active Directory, VPN), and persistent data stores (PostgreSQL, Redis).

9.2 Architectural Zones

Zone	Responsibility	Key Design Decision
Client Layer	Role-separated portal surfaces	Single Next.js app; portal views gated by JWT role
API Gateway	Auth, authorisation, rate limiting, injection guard	All cross-cutting concerns resolved before business logic executes
AI Orchestration	Multi-step reasoning, planning, tool execution	LangGraph StateGraph provides deterministic routing between AI nodes
Deterministic Services	Classification, enrichment, SLA, approval	AI output is never directly submitted to external systems — always validated first
Integration Layer	Enterprise system adapters	Adapter pattern with mock/live toggle per environment variable
Data Layer	Persistence and caching	SQLAlchemy ORM; database-agnostic for dev/prod parity

10. AI Architecture

10.1 Purpose & AI System Responsibilities

The AI subsystem performs four distinct functions: natural language intent recognition, diagnostic interview planning, root-cause hypothesis reflection, and ITSM taxonomy classification. To eliminate non-deterministic failure modes, AI components operate within strict boundaries.

10.2 AI Provider Fallback Architecture

To prevent platform outage during primary LLM rate-limiting or provider failure, the AI abstraction layer implements a three-tier resilient fallback model.

Figure 5 — AI Provider Three-Tier Fallback Architecture

Figure 5 illustrates the multi-provider failover strategy implemented behind the BaseAIProvider abstraction. Inbound requests are first routed to Google Gemini 2.5 Flash for high-speed intent detection and tool planning; upon API timeout

or rate-limit exceptions, traffic seamlessly fails over to Anthropic Claude 3 Sonnet. If all external LLM services become unavailable, the pipeline falls back to a deterministic 11-domain keyword regular expression engine to maintain triage availability.

10.3 AI Responsibilities by Node

Node	AI Role	Deterministic Gate
Router	Classify message as CHAT or TROUBLESHOOT	Exact string match on output
Context Router	Route to correct pipeline branch	Output constrained to four enum values
Intent	Classify IT issue category	Validated against valid_categories from incident_config.json
Diagnostic Interview	Determine information sufficiency; generate targeted questions	Short-circuit to END if questions are pending
Planner	Generate ordered tool execution plan	Plan validated against registered tool registry
Knowledge	Retrieve and rank relevant KB articles	Top-N selected by relevance score
Tool	Select and execute enterprise tool	Tool result validated before state update
Multi-Step	Evaluate tool results; decide whether to loop or escalate	troubleshooting_complete boolean controls loop exit
Root Cause	Synthesise hypothesis from tool chain results	Structured JSON output validated by Pydantic
Reflection	Evaluate reasoning quality; flag low-confidence decisions	Reflection score used by Decision node
Decision	Choose: CREATE_TICKET, EXECUTE_ACTION, or ASK_MORE_INFO	Enum output; unknown values default to ASK_MORE_INFO
Approval	Evaluate whether action requires approval	Approval status: PENDING, APPROVED, REJECTED
Classification	Assign ITSM category, subcategory, assignment group, CI	Validated against live ServiceNow metadata before submission

Table 5 — AI Node Responsibilities and Deterministic Gates

10.4 Classification Service Design

The ClassificationService acts as the boundary between AI output and ServiceNow incident creation. It enforces confidence thresholds (HIGH >= 0.90, MEDIUM 0.70-0.89, LOW < 0.70) and routes responses through Pydantic schema validation before payload construction.

11. LangGraph Workflow

11.1 StateGraph Architecture

The core AI reasoning pipeline is implemented as a 23-node LangGraph StateGraph (`backend/app/graph/graph.py`). The state graph enforces TypedDict schema validation across every turn and decouples node logic from graph routing.

11.2 AgentState Schema & Field Groups

Field Group	Fields
Session Context	session_id, username, user_role, user_message
Routing	decision, route, intent
AI Outputs	root_cause_analysis, reflection, plan, knowledge_context, tool_result
Approval	approval_required, approval_status, approval_action
Ticket	ticket, ticket_lifecycle, assigned_team
Multi-Step	tool_chain, hypothesis_tracker, troubleshooting_iterations, troubleshooting_complete, next_tool
Diagnostic	diagnostic_interview, conversation_goal, active_issue
SLA	sla, notifications

Table 6 — AgentState TypedDict — Field Groups

11.3 23-Node StateGraph Topology

The 23-node state graph controls execution flow through conditional edge routing based on conversation state, intent classification, and diagnostic completion.

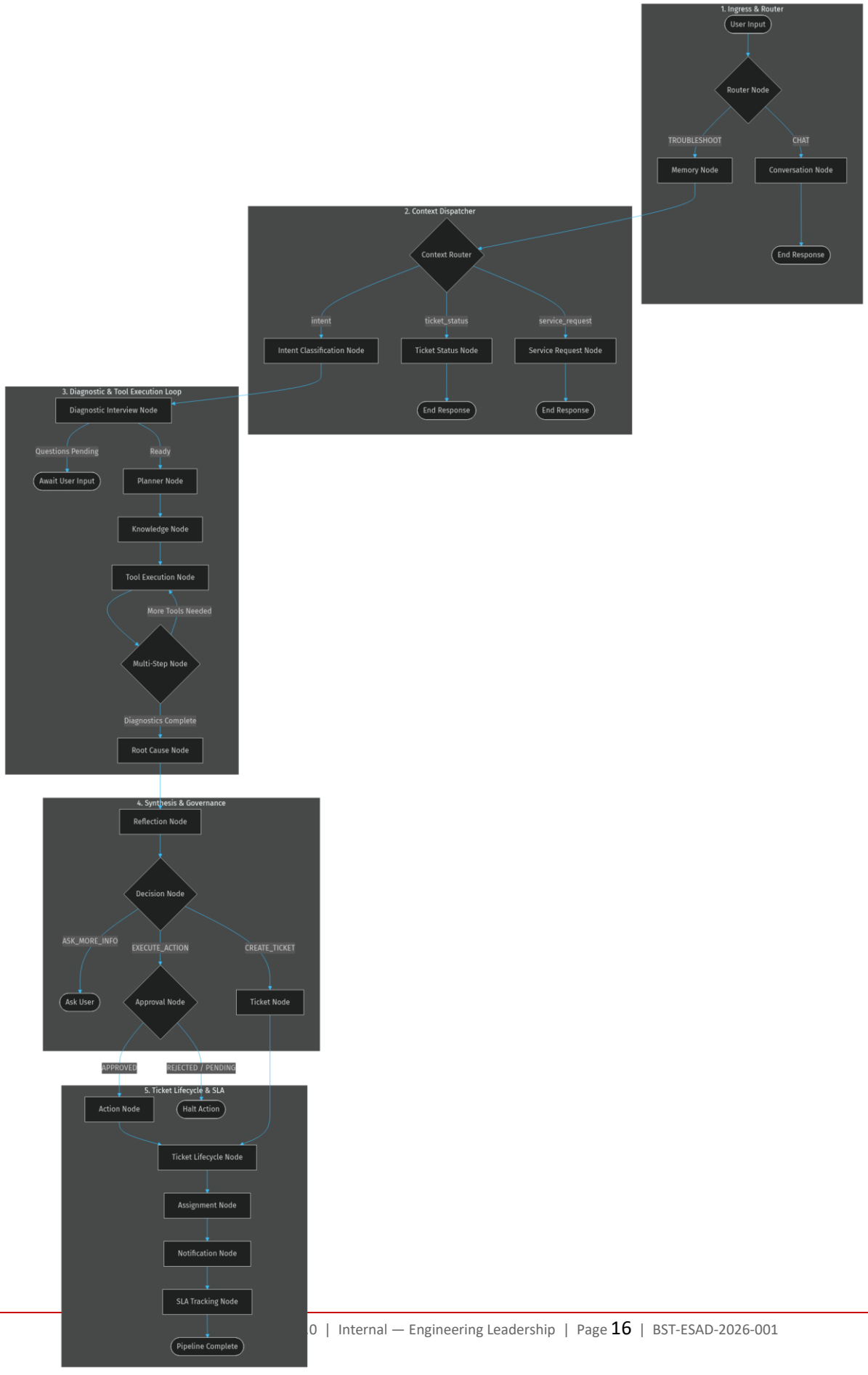
1. Ingress & Router
User Input
Router Node
TROUBLESHOOT
CHAT
Memory Node
Conversation Node
End Response
2. Context Dispatcher
Context Router
intent
ticket_status
service_request
Intent Classification Node
Ticket Status Node
Service Request Node
End Response
End Response
3. Diagnostic & Tool Execution Loop
Diagnostic Interview Node
Questions Pending
Ready
Await User Input
Planner Node
Knowledge Node
Tool Execution Node
More Tools Needed
Multi-Step Node
Diagnostics Complete
Root Cause Node
4. Synthesis & Governance
Reflection Node
Decision Node
ASK_MORE_INFO
EXECUTE_ACTION
CREATE_TICKET
Ask User
Approval Node
Ticket Node
APPROVED
REJECTED / PENDING
5. Ticket Lifecycle & SLA
Action Node
Halt Action
Ticket Lifecycle Node
Assignment Node
Notification Node
SLA Tracking Node
Pipeline Complete

Figure 6 — LangGraph 23-Node Workflow — Conditional State Graph Topology

Figure 6 details the full 23-node state machine governing the AI agent execution lifecycle. Inbound interactions enter through the Router node, which separates conversational chat from diagnostic troubleshooting. The multi-step diagnostic loop iteratively invokes system tools (VPN, Active Directory, Ping) and routes through Root Cause and Reflection nodes before rendering terminal decisions (Ticket Creation, Action Execution, or User Clarification).

11.4 Conditional Routing Logic

Conditional Edge	Routing Logic
route_after_router	decision == 'CHAT' → conversation; otherwise → memory
context_router	Lambda on state['route']: ticket_lifecycle | ticket_status | service_request | intent
route_after_diagnostic_interview	decision_response populated → END; else → planner
route_after_multi_step	troubleshooting_complete is False and next_tool is not None → tool; else → root_cause
route_decision	CREATE_TICKET → ticket; EXECUTE_ACTION → approval; else → END
route_after_approval	approval_status == 'APPROVED' → action; else → END
route_after_lifecycle	ticket['ticket_id'] exists → assignment; else → notification

Table 7 — LangGraph Conditional Edge Routing Logic

12. Conversation Lifecycle

12.1 Session State & Memory Persistence

Conversations are keyed by session_id in the platform database (`conversations` and `conversation_events` tables). The Memory node fetches dialogue history prior to pipeline execution and serializes updated state context upon completion.

12.2 Diagnostic Interview & Information Sufficiency

The Diagnostic Interview node evaluates whether sufficient contextual details (device ID, error codes, network location) are present before diagnostic planning begins. If required fields are missing, targeted clarification questions are rendered to the employee, pausing pipeline execution until the next turn.

13. ServiceNow Integration Architecture

13.1 Adapter Pattern & ServiceNow Client

Enterprise integrations implement an adapter design pattern with explicit mock/live environment toggles. `ServiceNowClient` manages OAuth2 Password Grant tokens with thread-safe RLock refresh protection.

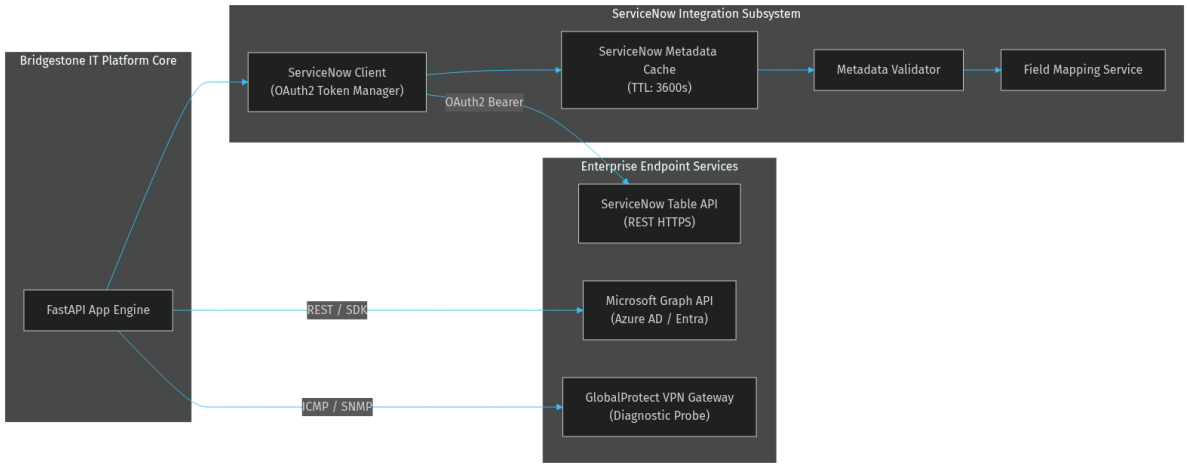

Figure 7 — Enterprise Integration Adapter Topology

Figure 7 illustrates the system integration topology connecting the core FastAPI backend with enterprise endpoints. ServiceNow interactions pass through a thread-safe token manager and a 3600-second metadata cache. External API calls to ServiceNow Table API, Microsoft Graph (Entra ID), and GlobalProtect VPN gateways are abstracted behind unified interfaces that toggle between live HTTPS calls and mock fixtures based on environment settings.

13.2 ServiceNow Metadata Validation Pipeline

To prevent invalid field submission errors when creating incidents, the platform routes AI outputs through a rigorous 4-stage validation pipeline.

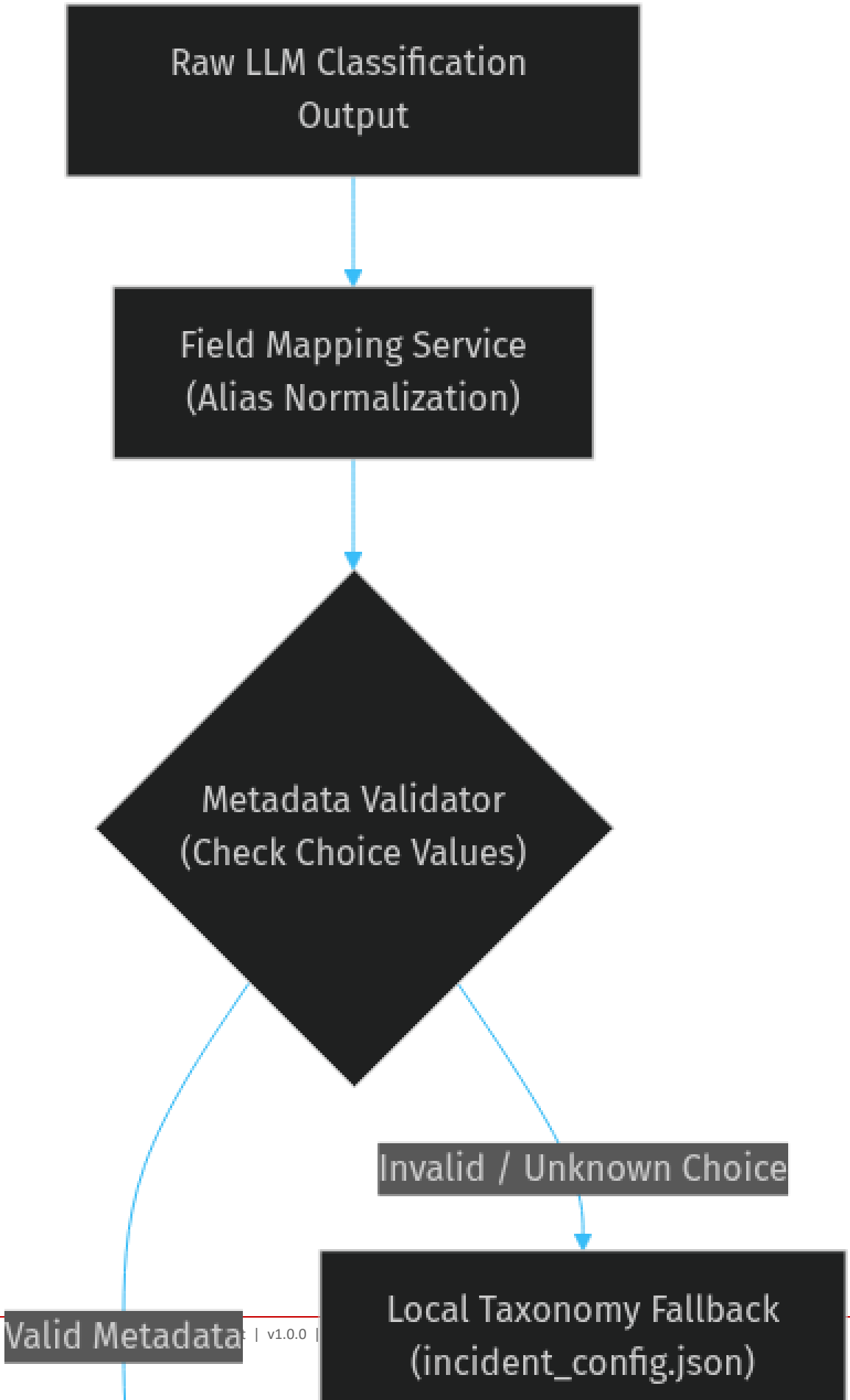
Raw LLM Classification Output
Field Mapping Service (Alias Normalization)
Metadata Validator (Check Choice Values)
Invalid / Unknown Choice
Local Taxonomy Fallback (incident_config.json)
Valid Metadata

Figure 8 — ServiceNow Metadata Validation Pipeline

Figure 8 outlines the deterministic validation chain applied to LLM classification outputs before invoking ServiceNow REST APIs. Raw AI predictions are first mapped to canonical ServiceNow choice values by the FieldMappingService using `incident_config.json`. The MetadataValidator checks choice existence against cached instance choices; if an invalid choice is detected, local taxonomy defaults apply before IncidentEnrichmentService computes impact, urgency, and priority values.

13.3 ServiceNow Incident Lifecycle

Platform Status	ServiceNow Trigger / Action	Valid Next Status
NEW	Platform creates INC via OAuth2	IN_PROGRESS or WAITING_MANAGER
WAITING_MANAGER	SOFTWARE_INSTALLATION / privileged request	READY_FOR_ADMIN or REJECTED
READY_FOR_ADMIN	Manager approves	ACCESS_GRANTED
ACCESS_GRANTED	Admin grants LAPS credentials	COMPLETED
IN_PROGRESS	Assigned to team	RESOLVED
RESOLVED	Engineer resolves	CLOSED (auto)
CLOSED	Final state	—
REJECTED	Manager rejects request	CLOSED

Table 8 — ServiceNow Incident Lifecycle — Platform Status States

14. Security Architecture

14.1 Defence-in-Depth Layer Stack

The security model implements strict defence-in-depth across perimeter, transport, application, and compliance audit layers.

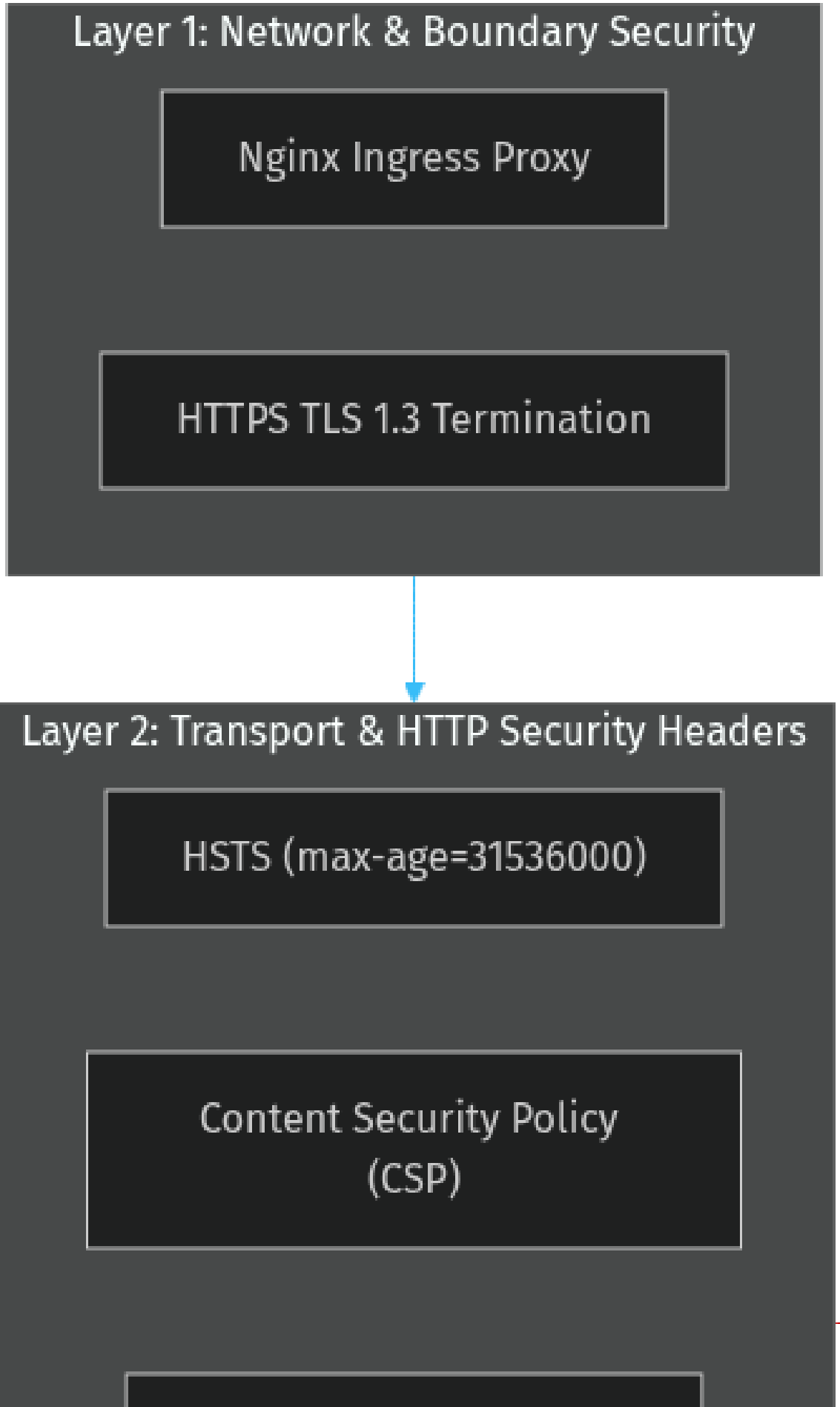
Layer 1: Network & Boundary Security
Nginx Ingress Proxy
HTTPS TLS 1.3 Termination
Layer 2: Transport & HTTP Security Headers
HSTS (max-age=31536000)
Content Security Policy
(CSP)

Figure 9 — Security Defence-in-Depth Layered Architecture

Figure 9 details the 4-layer security architecture enforcing corporate data protection standards. Perimeter TLS 1.3 termination at Nginx is fortified by transport-layer HTTP security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options). Application middleware enforces 512 KB request payload limits, sliding-window rate limiting, PromptGuard injection scanning, and JWT RBAC checks, while all state modifications append to an immutable SHA-256 audit log.

14.2 Authentication & Authorization Flow

User authentication utilizes bcrypt password hashing and PyJWT HS256 token issuance. Every endpoint verifies token claims and role authorization via `RoleChecker` dependencies.

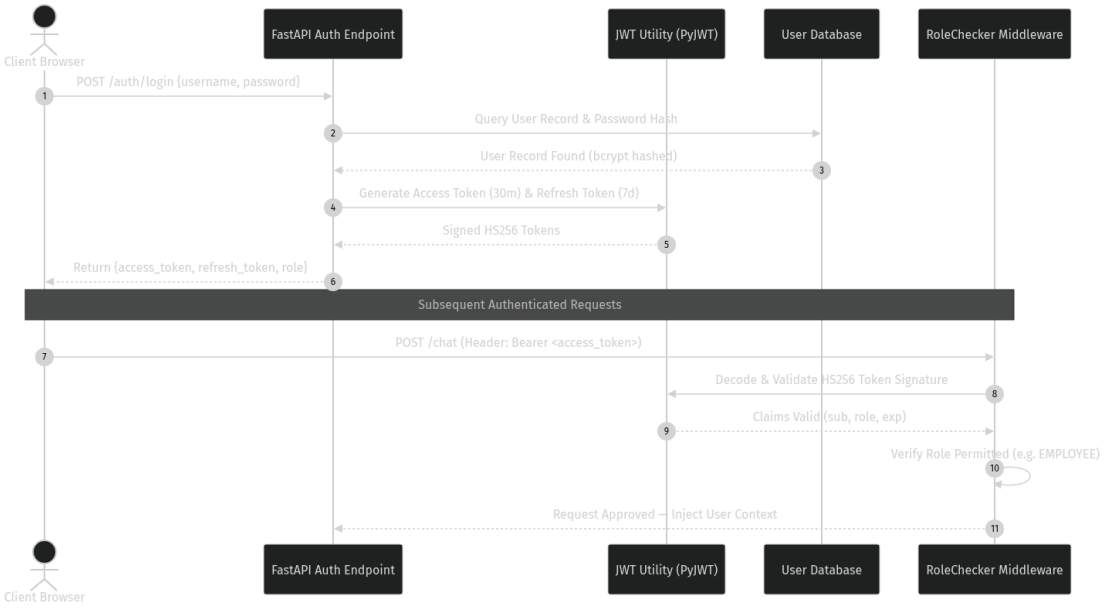

Figure 10 — Authentication and Authorisation Sequence Flow

Figure 10 demonstrates the credential verification and Bearer token lifecycle. Upon successful `/auth/login` authentication against bcrypt password hashes in the database, the API issues a 30-minute access token and a 7-day refresh token. Subsequent request headers are parsed by FastAPI `oauth2_scheme`, verifying HS256 signatures, token expiry, and user role entitlements prior to pipeline invocation.

14.3 Role-Based Access Control (RBAC)

Role	Permitted Endpoints
EMPLOYEE	/chat, /my-tickets, /auth/me
MANAGER	/chat, /manager/tickets/*, /analytics/*, /auth/me
ADMIN	/tickets/*, /admin/queue, /admin/tickets/*, /analytics/*, /knowledge/*, /auth/me
SUPERADMIN	All endpoints

Table 9 — Role-Based Access Control Model

14.4 PromptGuard Injection Classifier

PromptGuard protects the LLM pipeline against prompt injection attacks using full-phrase anchored regular expressions.

Risk Level	Action	Pattern Examples
HIGH	HTTP 400 + ImmutableAuditLog entry	'ignore all instructions', 'you are now DAN', 'forget your previous instructions'
MEDIUM	Sanitise fragment — pipeline continues	'act as X without restrictions', 'from now on you must', [INST] markers
LOW	Allow — emit structured warning log	Ambiguous phrasing partially matching medium patterns

Table 10 — PromptGuard Three-Tier Injection Classification

14.5 Immutable Audit Logging

Compliance auditability is maintained by `ImmutableAuditLog`, which exposes only an append-only `record()` method. Every entry captures correlation ID, actor identity, action type, timestamp, and a SHA-256 hash of the JSON payload.

15. Database Architecture

15.1 Data Model Overview

The database tier encompasses 19 SQLAlchemy ORM models organized across Identity, ITSM Workflow, AI Pipeline, Compliance Audit, SLA Tracking, and Platform Operations domains.

15.2 Core Entity Relationship Diagram

Primary data models maintain relational integrity across user identities, tickets, approval records, and SLA event histories.

Figure 11 — Core Database Entity Relationship Diagram

Figure 11 defines the core entity relationship schema governing the relational database. User records maintain 1-to-many relationships with Ticket entities. Each Ticket tracks historical lifecycle changes across SlaEscalationHistory, SlaAuditEvent, ApprovalHistory, TicketTimeline, and TicketComment tables, while AuditLog and RbacAuditLog maintain unconstrained append-only audit structures.

15.3 Core Entities Table

Entity	Table	Key Fields
User	users	id, username (UK), role, hashed_password, is_active
Ticket	tickets	ticket_id (UK), category, status, request_type,

		servicenow_id, servicenow_number, sla_state, approval_status, laps_*
AuditLog	audit_logs	correlation_id, event_type, actor, payload_hash, payload_json, timestamp
SlaEscalationHistory	sla_escalation_history	ticket_id (FK), escalation_level, notified_recipients, escalated_at
SlaAuditEvent	sla_audit_events	ticket_id (FK), event_type, sla_pct, sla_state, event_at
RbacAuditLog	rbac_audit_logs	username, role, action, resource, outcome, correlation_id, timestamp
ApprovalHistory	approval_history	ticket_id (FK), action, performed_by, notes, performed_at
Conversation	conversations	session_id (UK), username, history_json, created_at, updated_at
KnowledgeDraft	knowledge_drafts	title, root_cause, resolution, category, confidence, status, source_ticket_id (FK)

Table 11 — Core Database Entities — Key Fields

16. API Request Lifecycle

16.1 End-to-End Chat API Request Lifecycle

Inbound HTTP requests to `/chat` traverse sequential middleware components prior to AI state graph invocation.

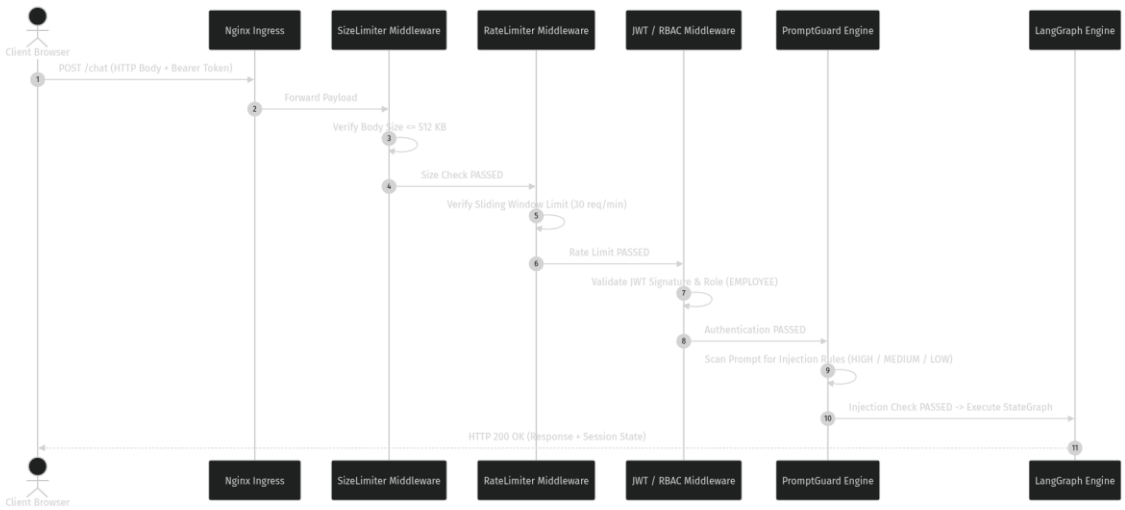

Figure 12 — Chat Endpoint API Request Lifecycle Sequence

Figure 12 traces the multi-layered execution path of an incoming POST `/chat` payload. Request execution begins at Nginx, moving sequentially through RequestSizeLimitMiddleware (<=512 KB), RateLimiter (30 req/min sliding window),

JWT Auth & RoleChecker dependencies (EMPLOYEE role verification), and PromptGuard injection scanning. Only payloads successfully clearing all validation gates enter the LangGraph execution engine.

16.2 Middleware Failure Handling

Layer	Failure Response	Audit Action
Request Size Limit	HTTP 413	Warning log
Rate Limiter	HTTP 429 + Retry-After header	Warning log
JWT Auth	HTTP 401	SECURITY_UNAUTHORIZED_TOTAL metric incremented
Role Checker	HTTP 403	RBAC audit log entry + SECURITY_PERMISSION_DENIED_TOTAL
PromptGuard HIGH	HTTP 400	ImmutableAuditLog entry with matched pattern
PromptGuard MEDIUM	Request sanitised — pipeline continues	Structured warning log emitted

Table 12 — Middleware Chain Failure Responses

16.3 Full API Surface

Method	Endpoint	Role	Description
POST	/auth/login	Public	Issue JWT access + refresh tokens
GET	/auth/me	Authenticated	Return current user identity and role
POST	/chat	EMPLOYEE	Submit message to AI pipeline
GET	/my-tickets	EMPLOYEE	List user's own tickets
GET	/tickets	ADMIN, MANAGER	List all tickets
GET	/tickets/{id}	ADMIN, MANAGER	Get full ticket detail
PUT	/tickets/{id}/status	ADMIN	Update ticket status
GET	/manager/tickets	MANAGER, ADMIN	Pending approval queue
POST	/manager/tickets/{id}/approve	MANAGER	Approve privileged request
POST	/manager/tickets/{id}/reject	MANAGER	Reject privileged request
GET	/admin/queue	ADMIN	Admin action queue
POST	/admin/tickets/{id}/grant-access	ADMIN	Grant temporary admin access (LAPS)
POST	/admin/tickets/{id}/complete	ADMIN	Mark ticket complete
GET	/analytics/*	ADMIN, MANAGER	Analytics and reporting endpoints
GET	/knowledge/*	ADMIN	Knowledge base management

GET	/health	Public	Liveness probe
GET	/metrics	Internal	Prometheus metrics scrape endpoint

Table 13 — Full API Surface

17. Deployment Architecture

17.1 Production Docker Topology

Production deployment utilizes Docker Compose with isolated container networks, persistent named volumes, and container health check dependencies.

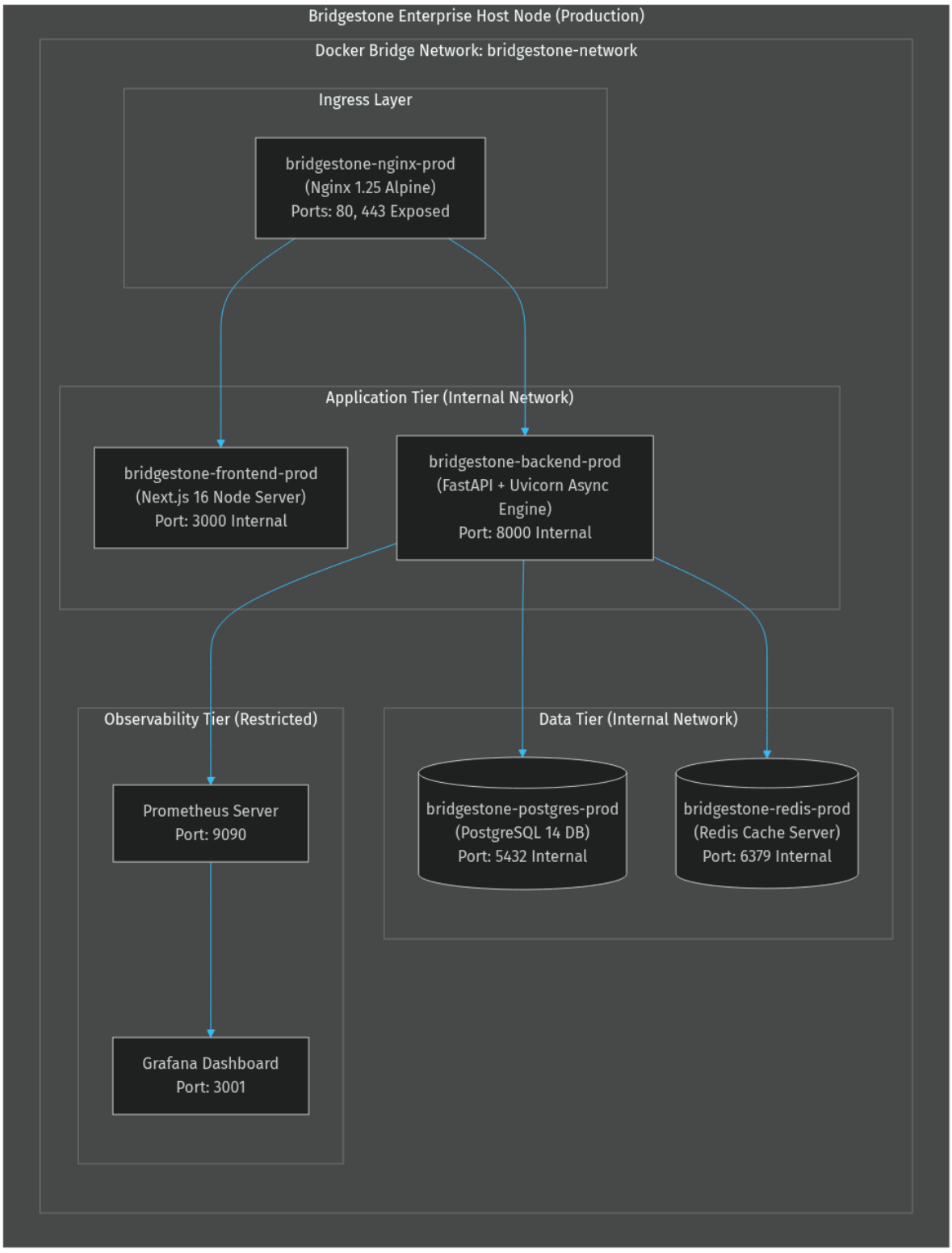

Figure 13 — Docker Compose Production Deployment Topology

Figure 13 illustrates the containerized service composition operating within the isolated `bridgestone-network` bridge. Nginx acts as the single external gateway exposing ports 80 and 443. Next.js frontend and FastAPI backend containers

communicate over internal ports, depending on PostgreSQL and Redis health checks before accepting incoming connections.

17.2 Environment Matrix

Environment	Database	ServiceNow	Graph / AD	Compose File
Development	SQLite (local)	Mock	Mock	docker-compose.dev.yml
QA	PostgreSQL (container)	Mock	Mock	docker-compose.qa.yml
Backend-only	SQLite (local)	Mock	Mock	docker-compose.backend-only.yml
Production	PostgreSQL (container, persistent volume)	Live	Live	docker-compose.prod.yml

Table 14 — Environment Matrix

18. Monitoring & Observability Architecture

18.1 Observability Architecture & Metrics

The observability framework integrates Prometheus metric scraping (`/metrics`), Grafana dashboards, structured JSON logging, and OpenTelemetry-compatible tracing spans.

18.2 Prometheus Metric Categories

Category	Key Metrics
HTTP	http_requests_total, http_failures_total, http_request_duration_seconds
Security	security_logins_total, security_failed_logins_total, security_permission_denied_total, security_jwt_expired_total
LLM	llm_requests_total, llm_success_total, llm_failure_total, llm_latency_seconds
AI Agent	agent_execution_duration_seconds [label: agent_name]
Pipeline	pipeline_span_duration_seconds [labels: service, provider, status]
Database	db_connections_active
SLA	sla_escalation_total, sla_breaches_total
ServiceNow	servicenow_api_calls_total, servicenow_api_errors_total

Table 15 — Prometheus Metric Categories

18.3 Background Jobs & APScheduler

Job	Schedule	Responsibility
sla_monitor_job	Every 60 seconds	Evaluate open tickets against SLA thresholds; transition SLA state machine
notification_job	Every 30 seconds	Dispatch pending notifications to recipients
auto_close_job	Every 60 seconds	Auto-close tickets in PENDING state beyond hold period
cleanup_job	Daily at midnight (0 0 * * *)	Purge expired sessions and stale temporary records

Table 16 — APScheduler Background Jobs

19. Technology Stack

19.1 Backend Stack

Component	Technology	Version	Justification
Runtime	Python	3.12	Latest stable; required by LangGraph and google-genai SDK
API Framework	FastAPI + Uvicorn	0.115	Async-native, OpenAPI auto-documentation, Pydantic v2 integration
AI Orchestration	LangGraph StateGraph	Latest	Deterministic, testable AI pipeline control flow with conditional edges
Primary AI	Google Gemini	gemini-2.5-flash	Large context window, function calling, developer free-tier access
Fallback AI	Anthropic Claude	Claude 3 Sonnet	Independent provider for resilience against Gemini outages
ORM	SQLAlchemy	2.x	Database-agnostic; SQLite and PostgreSQL parity
Auth	PyJWT + bcrypt	—	HS256 token issuance; bcrypt password hashing
HTTP Client	requests	—	Synchronous HTTP for ServiceNow REST API calls
Scheduling	APScheduler	—	In-process job scheduler with interval and cron triggers
Metrics	prometheus-client	—	Standard Prometheus Counter, Gauge, Histogram instrumentation
Validation	Pydantic v2	—	Schema enforcement on all AI outputs and

			API models

19.2 Frontend Stack

Component	Technology	Version	Justification
Framework	Next.js	16.2	App Router, SSR, API route proxying to backend
Runtime	React	19	Server Components, concurrent rendering
Language	TypeScript	5	Type safety across all four portal surfaces
Styling	Tailwind CSS	4	Utility-first; design system consistency
Animation	Framer Motion	12	Micro-animations for portal interactions
Charts	Recharts	3	Analytics dashboard visualisations
Icons	Lucide React	Latest	Consistent enterprise iconography

19.3 Infrastructure Stack

Component	Technology	Justification
Reverse Proxy	Nginx stable-alpine	TLS termination, path-based routing, upstream proxy
Database (prod)	PostgreSQL 14	ACID compliance, persistent volume, health check support
Cache	Redis	Session cache and ServiceNow metadata cache (production)
Containerisation	Docker + Compose	Environment-consistent deployment across dev, QA and production
Metrics Collection	Prometheus	Industry-standard scrape-based metrics
Dashboards	Grafana	Prometheus datasource; provisioning-as-code via volume mounts

20. Architecture Decision Records

Architecture Decision Records (ADRs) capture major architectural choices, design rationale, and accepted technical trade-offs.

ADR-001 — LangGraph StateGraph for AI Pipeline Orchestration

Field	Content
Status	Accepted

Context	The AI pipeline requires conditional branching, state persistence across nodes and multi-step tool-execution loops.
Decision	Use LangGraph StateGraph with typed AgentState TypedDict and conditional edge routing functions.
Consequences	Nodes are independently testable; routing logic is centralised in edge functions.

ADR-002 — Three-Tier AI Provider Fallback

Field	Content
Status	Accepted
Context	A single AI provider dependency creates a single point of failure.
Decision	Implement BaseAIProvider abstraction: Gemini (primary) → Claude (fallback) → keyword-rules deterministic engine.
Consequences	Guarantees operational continuity during provider outages.

ADR-003 — Configuration-Driven ITSM Taxonomy

Field	Content
Status	Accepted
Context	ServiceNow field values change when the instance is reconfigured.
Decision	All ITSM taxonomy externalised to incident_config.json, loaded at startup by ConfigurationService.
Consequences	Taxonomy changes require only JSON updates without Python code edits.

ADR-004 — In-Process Sliding-Window Rate Limiter

Field	Content
Status	Accepted
Context	External rate limiting dependencies add complexity for single-process setups.
Decision	Implement pure-Python _SlidingWindowStore using threading.Lock and collections.deque.
Consequences	Zero external dependencies for single-instance deployments.

ADR-005 — Adapter Pattern with Mock/Live Toggle

Field	Content
Status	Accepted
Context	Enterprise integrations require live credentials unavailable in dev/CI.
Decision	All integrations implement a BaseAdapter interface with mock toggles via env vars.
Consequences	Enables complete local offline pipeline testing.

ADR-006 — ServiceNow as Authoritative System of Record

Field	Content
Status	Accepted
Context	Without clear authority boundaries, ticket state could diverge.
Decision	Every platform ticket carries servicenow_id and servicenow_number. ServiceNow is authoritative.
Consequences	All incidents remain fully traceable to a ServiceNow INC number.

ADR-007 — Append-Only Immutable Audit Log

Field	Content
Status	Accepted
Context	Enterprise compliance requires tamper-evident audit logs.
Decision	ImmutableAuditLog exposes only record(). SHA-256 payload hashes stored alongside JSON payloads.
Consequences	Audit trail is non-repudiable under normal operation.

ADR-008 — Auto Schema Migration on Startup

Field	Content
Status	Accepted
Context	Maintaining Alembic migration chains adds friction during rapid POC iteration.
Decision	On startup, inspect schema and add missing columns via ALTER TABLE idempotently.
Consequences	Zero-friction schema evolution during development phases.

21. Engineering Challenges

21.1 AI Non-Determinism at ITSM Boundaries

Challenge	LLM outputs are stochastic. Category values produced by AI may fail ServiceNow validation.
Resolution	Four-stage validation chain: AI output → FieldMappingService → MetadataValidator → IncidentEnrichmentService before submitting API payloads.

21.2 Multi-Step Troubleshooting Loop Termination

Challenge	Tool execution loops must terminate deterministically without token budget exhaustion.
Resolution	AgentState tracks troubleshooting_iterations and troubleshooting_complete flag, enforcing loop exit conditions.

21.3 Concurrent ServiceNow OAuth2 Token Refresh

Challenge	Multiple concurrent requests could trigger duplicate token refresh calls.
Resolution	ServiceNowClient uses a threading.RLock gating refresh calls with a 60-second safety buffer.

21.4 ServiceNow Metadata Cache Consistency

Challenge	Instance reconfiguration can render choice values stale.
Resolution	TTL-based cache with graceful fallback to incident_config.json on API downtime.

21.5 Approval Workflow State Race Conditions

Challenge	Concurrent manager approval and admin access grants could cause inconsistent state.
Resolution	Strict pre-condition state checks in ApprovalService with full audit trail logging.

21.6 Prompt Injection Without Over-Blocking

Challenge	Keyword guards risk blocking valid IT prompts containing words like 'system' or 'act'.
Resolution	Full-phrase anchored regex patterns in PromptGuard ensuring accurate classification.

22. Future Roadmap

22.1 Near-Term (0–6 Months)

Initiative	Architectural Impact
Alembic Migration Chain	Replace auto-migration with versioned schema migrations
Redis-Backed Rate Limiter	Replace in-process store with Redis to support multi-replica scaling
Bi-Directional ServiceNow Sync	Webhook receiver for ServiceNow state alignment
Entra ID Live Integration	Replace mock AD adapter with production MS Graph OAuth2 flow
Knowledge Base Vector Search	Upgrade keyword retrieval to vector embeddings (pgvector)

22.2 Medium-Term (6–18 Months)

Initiative	Architectural Impact
Windows LAPS / CyberArk PAM	Replace mock credentials with live LAPS / PAM integration
Kubernetes Deployment	Multi-replica deployment with Redis shared state
OpenTelemetry Integration	Export traces to Jaeger, Datadog or Azure Monitor
Multi-Model AI Routing	Route requests to domain-optimised AI models

22.3 Long-Term (18+ Months)

Initiative	Architectural Impact
Predictive SLA Management	ML breach risk prediction model
Incident Pattern Clustering	Automated grouping of incidents into ServiceNow Problems
Multi-Tenant Architecture	Isolated data and AI contexts for subsidiaries

23. Conclusion

The Bridgestone IT AI Assistant establishes a production-credible architecture combining AI reasoning with deterministic enterprise controls, minimal operational complexity, and complete compliance auditability.

24. Appendix

24.1 ServiceNow Field Mapping Reference

Platform Concept	ServiceNow Field	Resolution Method
Category	category	AI → FieldMappingService → MetadataValidator
Subcategory	subcategory	AI → subcategory_map in incident_config.json
Assignment Group	assignment_group	assignment_group_map in incident_config.json
CMDB CI	cmdb_ci	ci_map in incident_config.json
Impact	impact	category_impact rules in incident_config.json
Urgency	urgency	category_urgency rules in incident_config.json
Priority	priority	Priority matrix derived from impact × urgency
Contact Type	contact_type	Static: 'Virtual Agent'
Caller	caller_id	Authenticated username from JWT

Table 17 — ServiceNow Field Mapping Reference

24.2 Environment Variables Reference

Variable	Default	Description
SECRET_KEY	dev default	JWT signing secret
DATABASE_URL	sqlite:///./bridgestone.db	SQLAlchemy connection string
REDIS_URL	redis://redis:6379/0	Redis connection (prod)
GEMINI_API_KEY	—	Google Gemini API key
ANTHROPIC_API_KEY	—	Anthropic Claude API key
SERVICENOW_INSTANCE_URL	—	ServiceNow instance base URL
SERVICENOW_USERNAME	—	ServiceNow API username
SERVICENOW_CLIENT_ID	—	OAuth2 client ID
SERVICENOW_CLIENT_SECRET	—	OAuth2 client secret
SERVICENOW_AUTH_TYPE	basic	Auth mode: basic or oauth2

SERVICENOW_METADATA_CACHE_TTL_SECONDS	3600	Metadata cache TTL
USE_MOCK_SERVICENOW	true	Disable live ServiceNow calls
USE_MOCK_GRAPH	true	Disable live MS Graph calls
USE_MOCK_AD	true	Disable live Active Directory calls
RATE_LIMIT_CHAT	30	Max chat requests per window
RATE_LIMIT_CHAT_WINDOW	60	Chat rate limit window in seconds
RATE_LIMIT_LOGIN	10	Max login attempts per window
RATE_LIMIT_ENABLED	true	Enable rate limiting
HSTS_ENABLED	false	Enable HSTS headers
MAX_REQUEST_SIZE_KB	512	Max request body size in KB

Table 18 — Environment Variables Reference

24.3 LangGraph Node Inventory — All 23 Nodes

#	Node	Source File	Responsibility
1	router	router_node.py	Classify message as CHAT or TROUBLESHOOT
2	conversation	conversation_node.py	Handle general conversational messages; end immediately
3	memory	memory_node.py	Load conversation history and active context into state
4	context_router	context_router_node.py	Route to appropriate pipeline based on conversation context
5	intent	intent_node.py	Classify IT issue into a diagnostic category
6	diagnostic_interview	diagnostic_interview_node.py	Conduct structured diagnostic questioning; short-circuit if questions pending
7	planner	planner_node.py	Generate ordered tool execution plan
8	knowledge	knowledge_node.py	Retrieve relevant KB articles and resolution precedents
9	tool	tool_node.py	Execute selected enterprise tool (VPN, AD, network, printer)
10	multi_step	multi_step_node.py	Evaluate tool results; decide whether to loop or proceed to root cause
11	root_cause	root_cause_node.py	Synthesise root cause from tool chain results
12	reflection	reflection_node.py	Evaluate reasoning quality and flag low-

			confidence decisions
13	decision	decision_node.py	Choose terminal action: CREATE_TICKET, EXECUTE_ACTION, or ASK_MORE_INFO
14	approval	approval_node.py	Evaluate approval requirement; set approval_status
15	action	action_node.py	Execute approved enterprise action
16	ticket	ticket_node.py	Classify, enrich, validate, create ServiceNow incident
17	ticket_lifecycle	ticket_lifecycle_node.py	Drive ticket state machine transitions (shared path)
18	ticket_status	ticket_status_node.py	Return current ticket status to employee
19	service_request	service_request_node.py	Handle service catalogue requests
20	assignment	assignment_node.py	Assign ticket to appropriate team based on category
21	notification	notification_node.py	Dispatch notifications to assigned recipients
22	sla	sla_node.py	Initialise or update SLA tracking state
23	context_router	context_router_node.py	Shared entry into ticket_lifecycle from standalone context path

Table 19 — LangGraph Node Inventory — All 23 Nodes

24.4 Database Model Inventory — All 19 Models

#	Model	Table	Domain
1	User	users	Identity
2	Session	sessions	Identity
3	Ticket	tickets	ITSM Workflow
4	ApprovalHistory	approval_history	ITSM Workflow
5	ActionHistory	action_history	ITSM Workflow
6	ServiceRequest	service_requests	ITSM Workflow
7	ServiceCatalog	service_catalog	ITSM Workflow
8	Conversation	conversations	AI Pipeline
9	ConversationEvent	conversation_events	AI Pipeline
10	AgentTrace	agent_traces	AI Pipeline

11	AuditLog	audit_logs	Compliance
12	RbacAuditLog	rbac_audit_logs	Compliance
13	SecurityEvent	security_events	Compliance
14	SlaAuditEvent	sla_audit_events	SLA
15	SlaEscalationHistory	sla_escalation_history	SLA
16	Notification	notifications	Operations
17	ScheduledJob	scheduled_jobs	Operations
18	KnowledgeDraft	knowledge_drafts	Knowledge
19	IncidentCluster	incident_clusters	Analytics

Table 20 — Database Model Inventory — All 19 Models

End of Document — BST-ESAD-2026-001 v1.0.0

Bridgestone IT Engineering — Platform Engineering — AI & Automation